Systems and Methods for Big and Unstructured Data - Notes - v0.1.0

260236

October 2026

Preface

Every theory section in these notes has been taken from the sources:

- Course slides. [1]

About:

GitHub repository

These notes are an unofficial resource and shouldn't replace the course material or any other book on systems and methods for big and unstructured data. It is not made for commercial purposes. I've made the following notes to help me improve my knowledge and maybe it can be helpful for everyone.

As I have highlighted, a student should choose the teacher's material or a book on the topic. These notes can only be a helpful material.

Contents

1 Big Data **4**
1.1 Big Data Landscape and the Data-Driven Value Cycle 4
1.1.1 The Big Data and AI Market Landscape 4
1.1.2 Technology evolution and the Hype Cycle 9
1.1.3 The Big Data Deluge 13
1.1.4 The Data-Driven Virtuous Cycle 17

Index **23**

1 Big Data

1.1 Big Data Landscape and the Data-Driven Value Cycle

1.1.1 The Big Data and AI Market Landscape

The first idea to establish is that **Big Data is not a single technology**. When people say "*Big Data*", they may be referring to databases, distributed storage, analytics platforms, cloud infrastructure, machine learning, visualization tools, or complete applications. In reality, these components form an **ecosystem**. A useful simplified picture is:

Data Sources → Data Infrastructure → Analytics → Applications

Different companies and technologies operate at different points of this pipeline.

Data sources: where everything starts

Before we can have "Big Data", something must generate data. Modern systems continuously produce information from many heterogeneous sources like web, smartphones, IoT devices, social networks, transactions, sensors, and more.

Example 1: E-Commerce data sources

For example, imagine an e-commerce service. It might collect: customer profiles, purchases, searches, clicks, page views, product reviews images, payment transactions, delivery information. Notice immediately that **these are not all the same kind of data**. Some may be highly structured:

```
order_id = 58321
customer_id = 912
price = 49.99
```

While others may be much less structured:

"*This product arrived late but works very well.*"

Or could be images, video, GPS coordinates, logs, etc. This heterogeneity will later become one of the defining properties of Big Data: **Variety**.

The infrastructure layer

Once data exists, we need technologies capable of **storing, transporting and processing it**. This is the infrastructure part of the Big Data market. Conceptually:

Raw Data → Infrastructure → usable data

Infrastructure contains things such as: distributed storage; databases; distributed computing systems; data ingestion systems; messaging systems; cloud infrastructure. Later in the notes we will encounter technologies belonging to

this general world, such as distributed databases and large-scale processing architectures.

The fundamental challenge is that a **traditional architecture** such as *one database on one machine* may eventually become **insufficient**. Then the architecture can evolve toward something like:

Machine 1	Machine 2	Machine 3
data	data	data

But in the other hand, a **distributed architecture** introduces new questions about partitioning, replication, consistency, failures, and scalability. These questions will be central later in the notes.

</> Open-source platforms

Another important part of the Big Data ecosystem is the strong presence of **open-source platforms**. Historically, much of the Big Data revolution was enabled by systems that made distributed storage and computation available without requiring every company to implement these mechanisms from scratch. Instead of an organization implementing distributed file storage, fault recovery, parallel computation, cluster scheduling, data partitioning from zero, it can **build on an existing platform**.

So we can think of these platforms as **providing a reusable technological foundation**:

$$\text{Distributed infrastructure} \rightarrow \text{platform} \rightarrow \text{data applications}$$

A famous historical example that we will study later is the *Hadoop* ecosystem.

Analytics layer

Simply storing huge quantities of data does not create value. Imagine a company has 100 TB of customer data. If nobody can derive useful information from it, those 100 TB are mostly a **cost**.

We therefore need another layer:

$$\text{Stored Data} \rightarrow \text{Analytics} \rightarrow \text{Information / Models / Predictions}$$

Analytics can include things such as: statistics; aggregation; data mining; machine learning; recommendation systems; forecasting; anomaly detection.

> **Example 2: Netflix analytics**
>
> Suppose Netflix records:
>
> - User A watched movie X
> - User A paused at 32 minutes

- User A finished movie Y
- User A abandoned movie Z

The **infrastructure problem** is: "*Can we reliably collect and store billions of these events?*". The **analytics problem** is: "*Can we infer something useful from them?*". For instance:

$$P(\text{user likes movie } m \mid \text{previous behaviour})$$

That distinction between **managing data** and **extracting information from data** is important.

Application layer

At the top, analytics becomes something the user or organization can actually use. For example:

$$\text{behaviour data} \rightarrow \text{recommendation model} \rightarrow \text{"Recommended for you"}$$

The user does not see the distributed storage system, cluster, database, or machine-learning infrastructure. They see the **application**. Other examples might include:

$$\text{traffic data} \rightarrow \text{traffic prediction} \rightarrow \text{route recommendation}$$

Or:

$$\text{financial transactions} \rightarrow \text{fraud detection model} \rightarrow \text{fraud alert}$$

Or:

$$\text{sensor readings} \rightarrow \text{predictive model} \rightarrow \text{maintenance warning.}$$

So there is a **transformation**:

$$\text{raw data} \rightarrow \text{processed data} \rightarrow \text{insight} \rightarrow \text{action}$$

Full data-stack providers

Not every company operates at only one layer. Some vendors provide an almost complete **data stack**. Instead of offering only a database, or only analytics, they might offer:

$$\text{storage} + \text{processing} + \text{analytics} + \text{ML} + \text{visualization} + \text{application services}$$

These are the **full data stack providers**.

Example 3

Cloud platforms are an intuitive modern example of this idea.
A single provider may offer services for: object storage, databases, stream

processing, distributed computing, data warehouses, ML training, model deployment, and business intelligence.

Therefore, the borders between the individual categories of the market are not strict.

🧠 Where does AI fit?

The AI market is strongly connected to this Big Data ecosystem because modern AI depends heavily on data infrastructure. A simplified relationship is:

$$\text{Data} \rightarrow \text{Data Engineering} \rightarrow \text{Machine Learning / AI} \rightarrow \text{Application}$$

For example, suppose we want to train a recommendation model. The ML algorithm itself might be only one piece. Before training it, somebody must:

1. Collect user interactions;
2. Clean them;
3. Integrate different sources;
4. Store them;
5. Transform them into suitable features;
6. Feed them efficiently to the model.

Thus AI requires a data system underneath it. A powerful model with poor data infrastructure is often practically useless.

≠ Big Data and AI are therefore not synonyms. This distinction is useful.

- **Big Data** is primarily concerned with questions such as: "*How do we collect, store, integrate, manage and process huge or complex datasets?*"
- **AI/ML** is more concerned with: "*How do we learn patterns or decision functions from data?*"

There is obviously a **large overlap** because AI systems often consume enormous quantities of data. But they solve different categories of problems. For Systems and Methods for Big and Unstructured Data (SMBUD), our main perspective is the **data-management side**. For example, before asking "*which neural network should we use?*", we are much more likely to ask:

- *Where does the data live?*
- *How is it represented?*
- *How is it partitioned?*
- *How do we query it?*
- *Can the system scale?*
- *What consistency guarantees do we have?*

Example 4: Spotify recommendation pipeline

Imagine Spotify wants to recommend songs. At the bottom we have **data sources**: play events, skips, likes, searches, playlists, artists, songs, devices, locations. These might generate billions of events. Then:

1. **Data Sources** (user events, song metadata, etc.) generate data.
2. **Infrastructure** collects and stores them: Distributed Storage, Databases, Streaming Systems.
3. **Analytics layer** looks for patterns: $P(\text{user } u \text{ likes song } s)$.
4. **Application** shows: Discover Weekly, Recommended songs, Similar artists.

Therefore:

User events → Big Data infrastructure → Analytics/AI → Product feat

This is essentially the market landscape represented as a technical pipeline.

Key Takeaways 1

We don't need to memorize the Big Data market as a collection of company logos. What matters is recognizing the **roles** inside the ecosystem:

$$\underbrace{\text{Data Sources}}_{\text{generate}} \rightarrow \underbrace{\text{Infrastructure}}_{\text{store/process}} \rightarrow \underbrace{\text{Analytics}}_{\text{understand/predict}} \rightarrow \underbrace{\text{Applications}}_{\text{create value}}$$

With **open-source platforms** providing many of the underlying technological building blocks and **full-stack providers** potentially spanning several or all of these layers.

And the AI market sits naturally on top of and alongside this ecosystem because without a scalable data pipeline, scalable AI is extremely difficult.

1.1.2 Technology evolution and the Hype Cycle

When a new technology appears, its adoption is rarely linear. A common pattern is:

$$\text{innovation} \rightarrow \text{hype} \rightarrow \text{disappointment} \rightarrow \text{maturity}$$

This is what the **Hype Cycle** tries to describe. The important point is that the perceived importance of a technology and its real practical maturity are **not the same thing**.

A technology can be talked about everywhere, heavily funded, considered revolutionary, while still being technically immature or poorly understood.

The basic shape

The Hype Cycle is usually represented as a curve with:

$$x = \text{time}, \quad y = \text{expectations}$$

The typical progression is:

1. **Technology Trigger**: a new technology appears, with prototypes and promising research results.
2. **Peak of Inflated Expectations**: enthusiasm grows, and people start expecting the technology to solve almost everything.
3. **Trough of Disillusionment**: reality catches up with expectations, and limitations become visible.
4. **Slope of Enlightenment**: people start understanding what the technology is actually good for, and real use cases emerge.
5. **Plateau of Productivity**: the technology becomes mature, and it reliably solves a known class of problems.

Let's understand each phase.

1. Technology Trigger

A new technology appears. At this point:

- There may be prototypes;
- Research results are promising;
- Companies start experimenting;
- Practical use cases are still limited.

The technology is interesting, but not yet mature. For Big Data, this kind of phase happened when ideas such as large-scale distributed data processing and new NoSQL systems began gaining attention. The important thing is that the potential is high, but evidence is still limited.

2. Peak of Inflated Expectations

Then enthusiasm grows. People start expecting the technology to solve almost everything. We often hear claims such as: "*this will completely replace existing systems*" or "*every company needs this*" or "*this changes everything*". Some early projects succeed, but many expectations are unrealistic. This is the **hype** phase.

For example, imagine a new database technology is introduced. Initially people may start saying: "*relation databases are obsolete*" or "*every dataset should be moved to this new system*". That is usually an **overreaction**. A new technology may be excellent for certain workloads without being universally superior.

And this point is especially relevant in SMBUD. Later, when we see MongoDB, Cassandra, Redis, Neo4j, Elasticsearch, vector databases, we should **not** think:

"newer technology" = "better technology"

Instead: technology suitability depends on requirements. For instance, a graph database is not "*better*" than a relational database in general. It may be much better for relationship-heavy queries, while a relational DBMS may be more appropriate for another application.

The Hype Cycle is therefore also a warning against **technology-driven design**. We should not choose a technology because it is fashionable.

3. Trough of Disillusionment

Eventually, reality catches up with expectations. Projects fail. Limitations become visible. People discover problems such as: high operational complexity; scalability limits; poor tooling; expensive maintenance; missing features; difficult migration; lack of expertise.

Enthusiasm then collapses. The technology may suddenly look much less revolutionary. This is the **Trough of Disillusionment**. But this does **not necessarily mean that the technology is bad**. Instead, unrealistic expectations are being corrected.

4. Slope of Enlightenment

At this stage, **people start understanding**: "*What is this technology actually good for?*". Instead of saying: "*use it everywhere*". The discussion becomes: "*use it when requirements X, Y, Z are present*". Real use cases emerge. Best practices become clearer. Tooling improves. Companies gain operational experience. This is when the technology starts becoming genuinely useful.

5. Plateau of Productivity

Eventually, the technology becomes mature. It is no longer interesting because it is fashionable. It is **interesting because it reliably solves a known class**

of problems. At this point: use cases are understood; benefits are measurable; limitations are known; tooling is mature; engineers know how to operate it. The technology has moved from *hype* to *engineering*. And that distinction is very important.

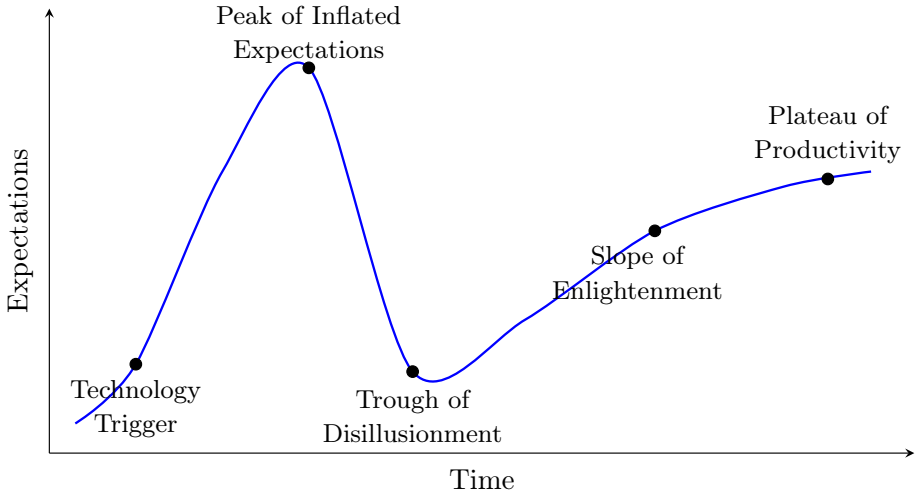


Figure 1: **The Hype Cycle.** The curve represents the evolution of expectations over time for a new technology.

Example 5: NoSQL

Consider how people may react when NoSQL systems appear. At the beginning:

"*Interesting alternative to relational databases*"

Then hype grows:

"*SQL is dead!*"

Eventually problems emerge:

- limited joins
- weaker consistency in some systems
- application complexity
- data duplication
- operational challenges

So enthusiasm decreases. Later, the industry develops a more nuanced understanding:

- Cassandra is useful for certain distributed, write-heavy workloads.

- Neo4j is useful when relationships and graph traversals dominate.
- MongoDB is useful when document-oriented structures align well with the application.
- PostgreSQL remains excellent for many transactional systems.

This mature view is exactly what we want in SMBUD.

The connection with the Big Data market

The reason the Hype Cycle appears right after the Big Data and AI market landscape is that markets around emerging technologies are extremely dynamic. When a technological wave appears:

new problem → new technologies → new companies → huge expectations

Many vendors enter the market. But some disappear, some merge, some technologies become standard, others remain niche, and others fail entirely. So the market landscape we saw earlier should not be interpreted as a fixed map. It **changes continuously as technologies mature**.

Why this matters for a data engineer

Imagine that our company says: "*Everyone is using vector databases. We should migrate everything to a vector database*". A good engineer should **not answer**: "*Yes, because vector databases are modern*". Instead, we ask:

What are our queries?
What type of data do we have?
What latency do we need?
What consistency guarantees?
What scale?
What problem are we solving?

Then we evaluate whether the technology fits those requirements. That is the engineering mindset the Hype Cycle encourages us to adopt.

Key Takeaways 2

The **Hype Cycle** is essentially a model of the gap between *expectations* and *actual technological maturity*. The typical evolution is: (1) Trigger, (2) Inflated Expectations, (3) Disillusionment, (4) Understanding, (5) Productive Use.
For us, the practical lesson is that we should not choose a technology because it is fashionable. Instead, we should choose it because its **data model, architecture, and trade-offs fit the problem we are trying to solve.**

1.1.3 The Big Data Deluge

The phrase **Big Data deluge** describes the idea that modern systems are producing data at a scale and rate that traditional data-management approaches were not designed for. The key idea is that data generation is growing faster than our ability to manage it traditionally.

The word *deluge* is deliberate: it suggests a flood. We are not just dealing with "*more rows in a database*". We are dealing with a continuous accumulation of data from a huge number of sources.

Where does all this data come from?

Historically, most digital data came from relatively controlled business systems such as: transactions, customer records, inventory, banking operations, and enterprise applications. These sources were usually structured and generated at a relatively predictable rate.

Today, data comes from many more places:

$$\text{Data Sources} = \text{Humans} + \text{Software} + \text{Machines} + \text{Sensors}$$

For example smartphone interactions, web clicks, GPS positions, social-media activity, server logs, IoT sensors, cameras, industrial machinery, payment systems, connected vehicles, and wearable devices. So the number of **data-producing entities** has increased dramatically.

Machines now generate data automatically

Traditional data was often created because a human explicitly performed a business action. For example:

$$\text{customer buys product} \rightarrow \text{transaction record}$$

Modern systems now generate much more data around the same action: user opened an app, searched for a product, scrolled the page, clicked an image, added the product to the cart, removed it, added it again, paid, recorded payment latency, updated delivery location, and opened a notification.

A single business event can now produce tens or hundreds of technical and behavioural events. Therefore:

$$\text{one real-world action} \rightarrow \text{many digital observations}$$

This is one reason why data volume has exploded.

Storage is no longer only about final business records

In a traditional information system, we might mainly keep the final result:

```
Order #1234
Customer = Alice
Total = € 84
```

But modern analytics often values the entire process that led to that result. For example search, click, view, pause, scroll, add-to-cart, remove-from-cart, purchase.

Why? Because those intermediate events reveal **behaviour**. The final transaction tells us: "*Alice bought product X*". But the complete event stream tells us much more: "*Alice searched three alternatives, compared prices, hesitated for five minutes, abandoned the cart, returned later, and then completed the purchase.*" That additional information can be far more useful for prediction and personalization.

From scarce data to abundant data

Historically, one difficulty was: "***how can we obtain enough data?***". Now, in many contexts, the problem has become: "***how can we manage all the data we already produce?***". This is a fundamental shift.

Previously, collecting data was expensive. Therefore organizations tended to decide carefully: "*which data should we store?*". Today, storage is relatively cheaper and automatic instrumentation is common, so the instinct often becomes: "*store as much as possible because it may become useful later*". This produces a huge expansion in stored information.

The deluge is not only about Volume

Although *deluge* immediately makes us think of **quantity**, the problem is broader. Imagine a smart city. It may collect traffic sensor readings, bus positions, weather measurements, CCTV video, social media, mobile phone signals, public-event data, parking information, and pollution sensors. The **challenge** is not merely that there are many bytes. The **data**:

- **Differs** in **type** (structured, semi-structured, unstructured);
- **Arrives** at different **rates** (from milliseconds to hours);
- **Has** different levels of **quality** (from highly accurate to noisy or missing).

This anticipates the Big Data dimensions we will later formalize as: Volume, Variety, Velocity, and Veracity. So the "*deluge*" is the phenomenon; the **Vs** (i.e., the four Vs) later give us a **framework for describing its properties**.

Why traditional systems start struggling

Suppose a company has a traditional architecture: one Relational DataBase Management System (DBMS). This may work perfectly well for a certain workload. But suppose the company begins ingesting 10^3 events per second. Then 10^5 and eventually 10^7 events per second. At the same time, it stores text, JSON, images, logs, user interactions, and sensor streams.

At some point, simply making one server larger becomes increasingly difficult or expensive. This creates pressure toward distributed storage and computation. Instead of one very large computer (**scale up**), we can distribute the work across many machines (**scale out**). This will later connect directly to horizontal scalability, Hadoop, distributed databases, sharding, and replication.

Data generation vs. useful information

One subtle point is that:

$$\text{more data} \neq \text{more knowledge}$$

A company might collect 1 PB of data and still learn almost nothing from it. The value appears only if the organization can transform that data. This gives us the pipeline:

$$\text{Data} \rightarrow \text{Processing} \rightarrow \text{Information} \rightarrow \text{Insight} \rightarrow \text{Action}$$

The deluge creates both: **a problem** because we need to manage enormous quantities of data, and **an opportunity** because that data potentially contains useful information.

Example 6: The Big Data deluge in practice

Consider a streaming platform. A very simple historical dataset might have been:

`user_id`, `movie_id`, `rating`, `date`

Now suppose the platform records:

- When playback starts
- When playback stops
- Pause events
- Rewind events
- Device type
- Network quality
- Search queries
- Time spent browsing

- Which thumbnail was shown
- Whether the thumbnail was clicked
- Percentage of episode watched

Instead of one record per movie interaction, we might now have hundreds of events. For one user, that might be 100 events per day. For 10^8 users, that is $100 \times 10^8 = 10^{10}$ events per day. That simple multiplication explains why distributed data architectures become necessary very quickly.

The feedback effect

Another important consequence is that successful digital applications often generate **even more data as they grow**. Suppose a service becomes popular, then:

1. More users use the service
2. Which produces more interactions
3. Which produces more data
4. Which may enable better analytics (assuming a good data pipeline)
5. Which improves the service and product
6. Which attracts more users

This already starts to look like a **virtuous cycle**, which is exactly what the next section will formalize.

Key Takeaways 3

The **Big Data deluge** means that digital systems generate and retain unprecedented amounts of data from a rapidly increasing number of sources.

It produces three important consequences.

1. Traditional centralized architectures may stop scaling.
2. We need new ways to store and process heterogeneous, fast-growing data.
3. The large amount of data creates opportunities for better analysis and decisions.

So the deluge is both a **problem** and an **opportunity**:

Data Deluge → Need process data → Extract insights → Create value

1.1.4 The Data-Driven Virtuous Cycle

The fundamental idea is that data should not simply be **collected and stored**. A data-driven organization continuously transforms data into value, and that value often creates new interactions that generate even more data. The cycle is:

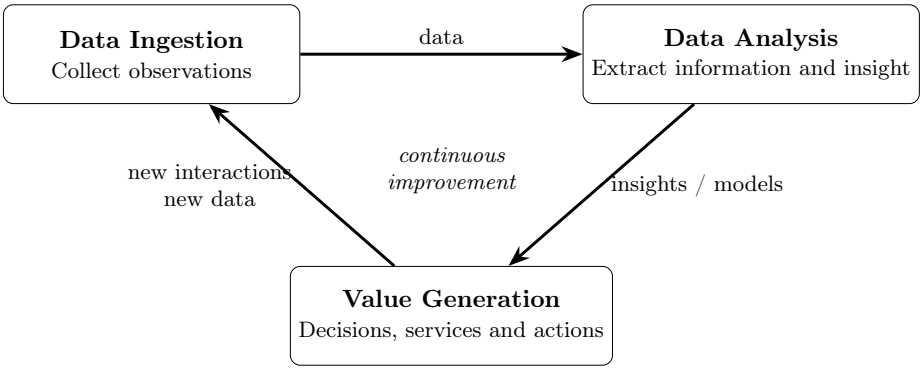


Figure 2: **The data-driven virtuous cycle.**

This is why it is called a **Virtuous Cycle** rather than just a pipeline. A pipeline would be:

$$A \to B \to C$$

Instead, the cycle is:

$$A \to B \to C \to A \to \cdots$$

The result of using data improves the system, which produces more and possibly better data, which can then be analyzed again.

Step 1: Data ingestion

The first step is **obtaining the data**:

$$\text{Data sources} \to \text{Data ingestion}$$

Data may originate from many independent systems such as WiFi logs, telecommunication networks, transport systems, IoT devices, social networks, websites, mobile applications, transactions, sensors, etc. For example, imagine a city's transportation system. It could continuously receive GPS positions of buses, ticket validations, traffic sensors, and weather information.

The job of ingestion is to **bring these observations into the data system so that they can eventually be processed**. Notice that ingestion is not the same as analysis. At this stage we are essentially saying "*something happened, record it*". For example:

```
15:32:04 - Bus 73 at position X
15:32:06 - passenger entered station Y
15:32:07 - traffic sensor reports congestion
```

Those events are data, but they are not yet an insight.

⇄ Ingestion can be batch or continuous. A useful distinction, which will become much more important later, is between data that arrives periodically and data that arrives continuously.

- **Batch**. Suppose a company receives a daily file: `sales-2026-09-24.csv` and processes it overnight. The flow is approximately:

  $$\text{collect during day} \rightarrow \text{process later}$$

  So the data is collected in a *batch* and processed later.

- **Streaming**. Now imagine sensor readings: $e_1, e_2, e_3, \ldots$ arriving continuously. We may want:

  $$\text{event} \rightarrow \text{processing} \rightarrow \text{result}$$

  Almost immediately.

That distinction will later lead us toward **batch processing versus stream processing**. For now, the important point is simply that ingestion gets data into the system, whether in batches or continuously.

🗠 Step 2: Data analysis

Collecting data alone produces no useful decision. So the second phase is:

$$\text{Data} \rightarrow \text{Analysis} \rightarrow \text{Information / Insight}$$

This is where we try to answer questions about the phenomena represented by the data. Exist two important kinds of analysis:

- **Descriptive analysis** asks "*what is happening, or what happened?*". For example:
  - *How many people entered a station today?*
  - *Which products sell the most?*
  - *At what time is traffic highest?*
  - *Which pages receive the most visits?*
  - *What percentage of users complete a purchase?*

  Mathematically, this often means computing statistics or aggregations over observed data. For example:

  $$\text{AverageDelay} = \frac{1}{n}\sum_{i=1}^{n} d_i$$

  We transform many individual observations $d_1, d_2, \ldots, d_n$ into useful information.

- **Predictive analysis** asks "*what is likely to happen?*". For example:

$$P(\text{congestion in 30 minutes} \mid \text{current observations})$$

Or:

$$P(\text{customer buys product } x \mid \text{past behaviour})$$

Now we are moving from observations of the past/present toward estimation of future or unknown outcomes. That can involve statistics, machine learning, forecasting, recommendation systems, etc.

Analysis is not valuable by itself. Suppose our analysis determines:

$$P(\text{traffic jam}) = 0.95$$

This is an insight, but if nobody uses that result, practically nothing changes. This leads to the third phase.

Step 3: Value generation

The output of analysis must eventually influence something:

$$\text{Insight} \rightarrow \text{Decision / Service / Action}$$

This is **value generation**. "*Value*" does not necessarily mean only money, but rather it may be **economic value**:

$$\text{better recommendations} \rightarrow \text{more purchases}$$

Or:

$$\text{better pricing} \rightarrow \text{higher margins}$$

But it can also be **social value**:

$$\text{traffic analysis} \rightarrow \text{better urban transport}$$

Or **cultural value**:

$$\text{analysis of cultural activity} \rightarrow \text{better cultural services}$$

Or **knowledge value**:

$$\text{data} \rightarrow \text{new understanding of a phenomenon}$$

So a *value* is not necessarily a *profit*. This is particularly important because Big Data is used by businesses, governments, researchers, healthcare systems, cities, and many other organizations.

Why is it a *virtuous* cycle?

Imagine an e-commerce company.

1. It **collects** searches, clicks, purchases, product views, and cart operations.

2. It **analyzes** the data and discovers that:

$$P(\text{buy headphones} \mid \text{previous laptop purchase})$$

   Is relatively high.

3. It **generates value** by recommending headphones to users who recently bought laptops.

If some users click and buy, that produces recommendation impressions, click, time spent, and purchases, which are **new data points**. The cycle starts again because the new data can be analyzed to improve future recommendations.

A more general representation

We can therefore write:

$$D_t \to A(D_t) \to V_t \to D_{t+1}$$

Where:

- D_t: data available at time t;
- $A(D_t)$: analysis performed on the data;
- V_t: value/action produced;
- D_{t+1}: new data resulting from subsequent interactions.

Ideally:

$$D_{t+1} > D_t$$

Not necessarily only in quantity, but also potentially in **quality and usefulness**. The organization can therefore improve iteratively.

Example 7: Navigation Application

Suppose a navigation service wants to estimate road congestion. First **collect**, the service receives GPS positions from users. Then **analyze**, it estimates road speed:

$$\text{road speed} = \frac{\text{distance}}{\text{time}}$$

For different road segments and detects congestion. Then **generate value**, it suggests a faster route to users.

But here's the interesting part. Users follow that route while still transmitting GPS data. Therefore the service receives additional information about: travel time, congestion, alternative routes, and changes in traffic. So:

$$\text{better service} \to + \text{ usage} \to + \text{ data} \to \text{better analysis} \to \text{better service}$$

That is a data-driven virtuous cycle.

Example 8: Recommendation System

Consider Netflix, which collects data about what users watch, skip, or pause. We can already anticipate the structure:

1. Viewing behaviour is collected;
2. Preferences are analyzed;
3. Content is recommended;
4. Users watch, skip, or pause;
5. New viewing behaviour is collected;
6. The preference model is improved;
7. Repeat.

The crucial distinction: data itself is not value

Having 100 TB of data does not mean we have 100 TB of value. Data becomes valuable only through a process:

$$\text{Data} \xrightarrow{\text{analysis}} \text{Insight} \xrightarrow{\text{application}} \text{Value}$$

Therefore, merely building an enormous data lake is not the objective. The real objective is to create a mechanism that continuously converts observations into useful actions.

Key Takeaways 4

The **data-driven virtuous cycle** consists of three fundamental steps:

$$\text{Collect / Ingest} \rightarrow \text{Analyze} \rightarrow \text{Build Value}$$

Where:

- **Data ingestion** acquires data from the available sources;
- **Data analysis** transforms observations into descriptions or predictions;
- **Value generation** uses the results to improve decisions, services, knowledge, or economic outcomes.

And the reason it is a **cycle** is that the value-generating actions create new interactions, which produce new data, which can be analyzed again.

References

[1] Brambilla Marco. Systems and methods for big and unstructured data. Slides from the HPC-E master's degree course on Politecnico di Milano, 2026.

Index

B

Big Data deluge 13

H

Hype Cycle 9

V

Virtuous Cycle 17